

UNIVERSIDADE PRESBITERIANA MACKENZIE

Cloud Developing — 2026/1

Gabriela Nellessen de Sousa — RA 10441930

**WATCHLIST DE FILMES E SÉRIES:
UMA APLICAÇÃO EM NUVEM NA AWS**

São Paulo

2026

Repositório: <https://github.com/gabinellessen/gabi-watchlist>

Vídeo de demonstração: <https://www.youtube.com/watch?v=Tmhgoa0qYRg>

1. Introdução

Para o projeto da disciplina, decidi montar uma watchlist de filmes e séries. A ideia veio de um problema bem pessoal: eu vivo esquecendo o que já assisti e o que ainda quero ver, então quis um sisteminha pra organizar isso, com nota e um espaço pra anotações.

Mais do que o aplicativo em si, o objetivo do trabalho era colocar essa aplicação pra rodar na nuvem usando vários serviços da AWS de forma integrada. Em vez de um CRUD rodando sozinho num servidor só, eu queria entender como separar as responsabilidades: a aplicação em contêineres, o banco de dados isolado e protegido, uma função serverless cuidando da parte de relatório, e tudo acessível por um único ponto de entrada.

2. Arquitetura

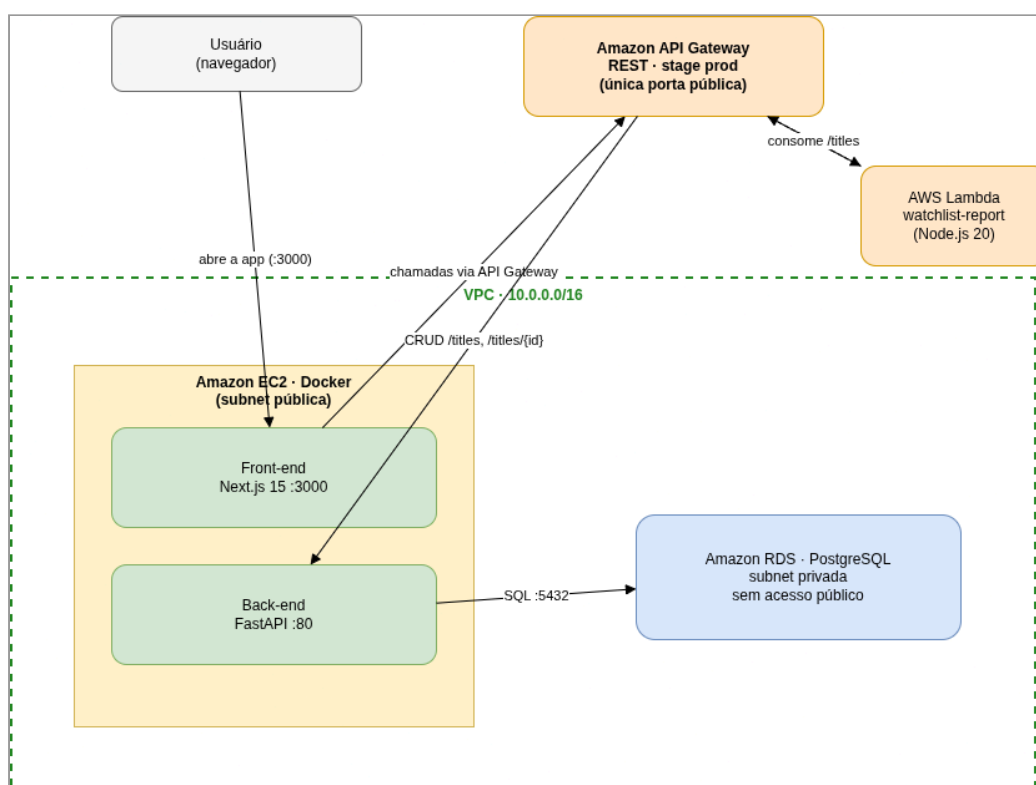


Figura 1 – Diagrama da arquitetura na AWS.

A aplicação está dividida em quatro componentes principais, todos dentro de uma VPC própria. Quem usa o sistema acessa o front-end, feito em Next.js, que roda em um contêiner Docker numa instância EC2. As telas não falam direto com o banco: toda requisição de dados passa pelo API Gateway, que é a única porta de entrada pública.

A partir do API Gateway, as rotas de cadastro, listagem, edição e remoção vão para o back-end — uma API em FastAPI, também em contêiner na EC2 — que é quem de fato conversa com o banco no Amazon RDS. Já a rota de relatório foi tratada de um jeito diferente: em vez de calcular as estatísticas no próprio back-end, ela é atendida por uma função Lambda. Essa função não toca no banco; ela chama a API pelo mesmo API Gateway, junta os dados e devolve os números prontos. Achei interessante fazer assim pra ver a parte serverless funcionando sem misturar com o resto.

A decisão que mais me deu trabalho foi a do banco. O RDS não podia ficar exposto, então coloquei ele numa sub-rede privada e configurei o Security Group pra aceitar conexão na porta 5432 apenas vinda do Security Group da instância EC2. Na prática, não existe caminho de fora da VPC que chegue no banco — e confesso que levei um tempo até essas regras ficarem certas.

3. Implementação

3.1 Back-end (FastAPI)

O back-end é uma API em Python com FastAPI e SQLAlchemy, conectada ao PostgreSQL. Uma coisa que facilitou bastante foi deixar a própria API cuidar de preparar o banco: quando o contêiner sobe, ela cria a tabela e já insere 12 títulos de exemplo. Assim eu não precisei rodar nenhum script SQL na mão depois de subir o RDS.

3.2 Front-end (Next.js)

O front é em Next.js 15 com TypeScript e Tailwind, com quatro telas: a lista de títulos, o formulário de cadastro, a tela de edição/exclusão e o relatório. O endereço do API Gateway entra no momento do build, então todas as chamadas já saem apontando pra ele.

Subir o front na EC2 foi onde travei mais tempo. Como usei uma instância pequena, o build do Next.js não cabia na memória e falhava no meio. Acabei resolvendo com uma área de swap na instância, e aí o build passou normalmente.

3.3 Função Lambda (/report)

A função é em Node.js e faz uma requisição à rota de listagem, através do API Gateway, para então agregar os dados: total de títulos, quantos de cada tipo e gênero, percentual já assistido, duração média e nota média. Como ela usa a API e não o banco, dá pra trocar o back-end de lugar que a Lambda continua funcionando.

4. Capturas dos serviços AWS

4.1 VPC e sub-redes

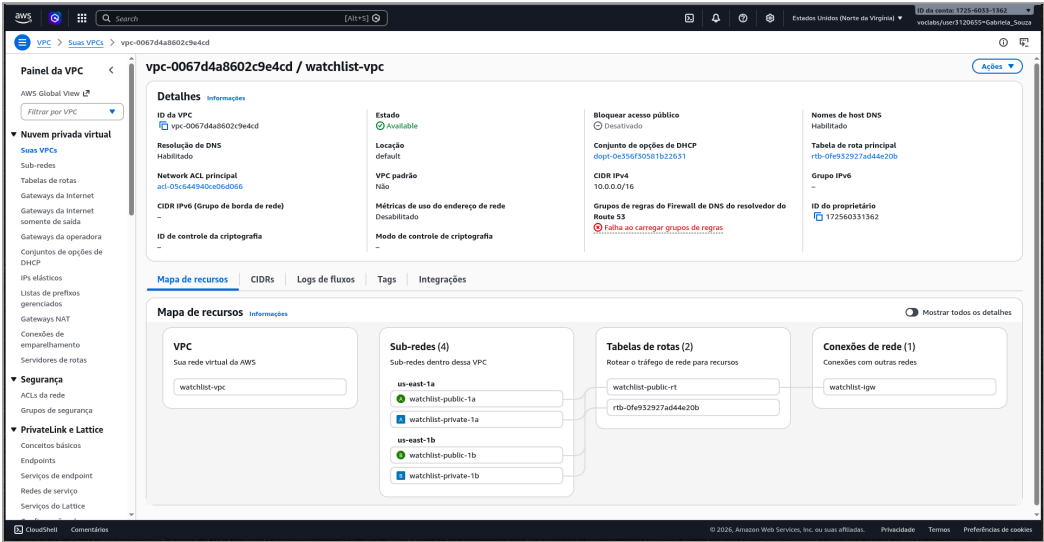


Figura 2 – VPC do projeto.

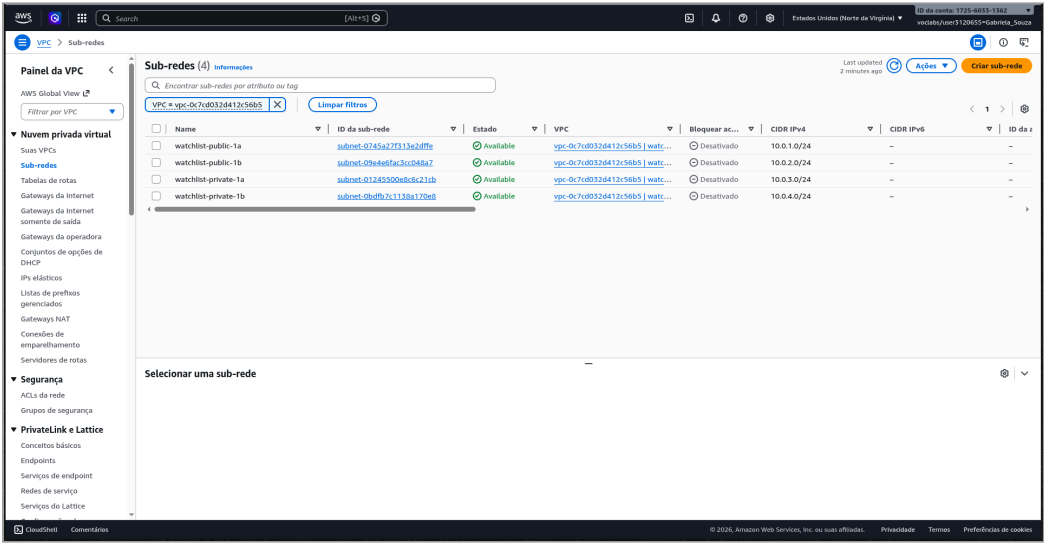


Figura 3 – Sub-redes (duas públicas e duas privadas).

4.2 Security Groups

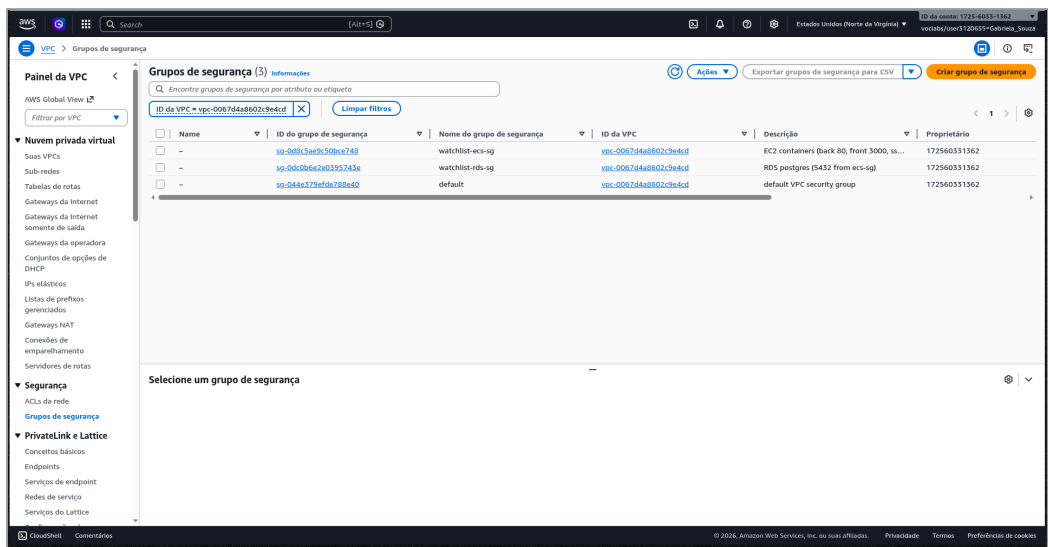


Figura 4 – Security Groups da EC2 e do RDS.

4.3 Amazon RDS

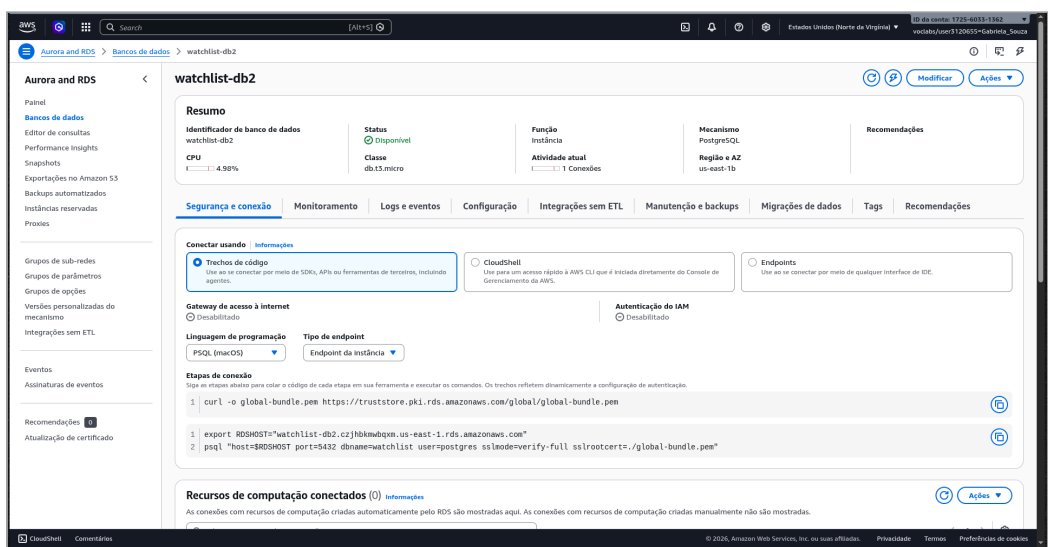


Figura 5 – Banco no RDS, com acesso público desabilitado.

4.4 EC2 e contêineres

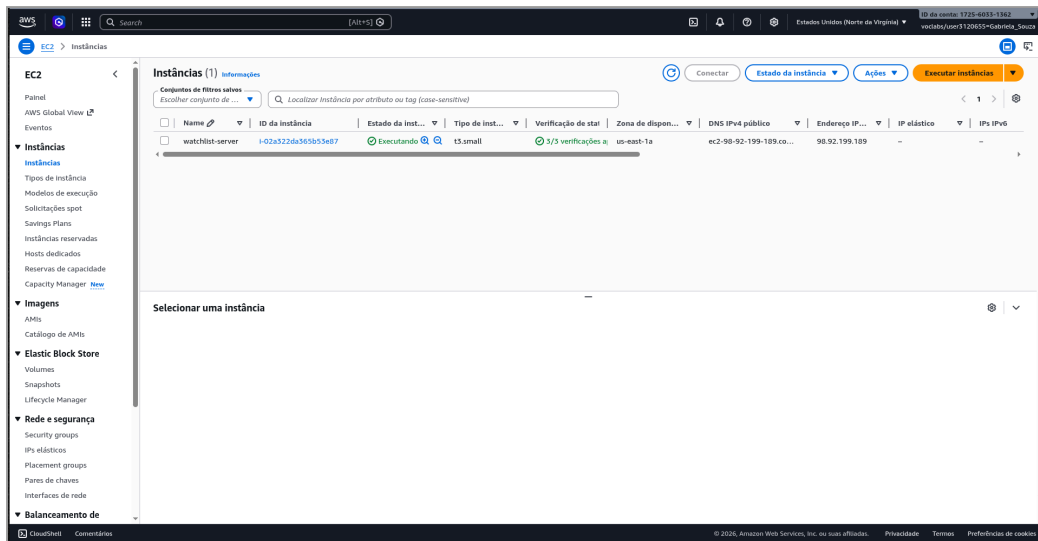


Figura 6 – Instância EC2 em execução.

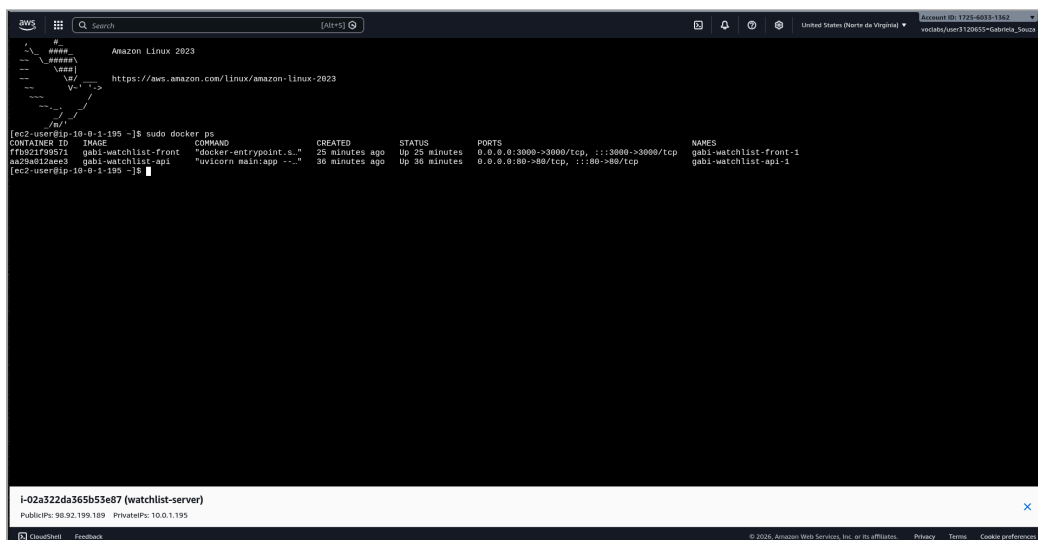


Figura 7 – Os dois contêineres (back-end e front-end) rodando na instância.

4.5 Lambda

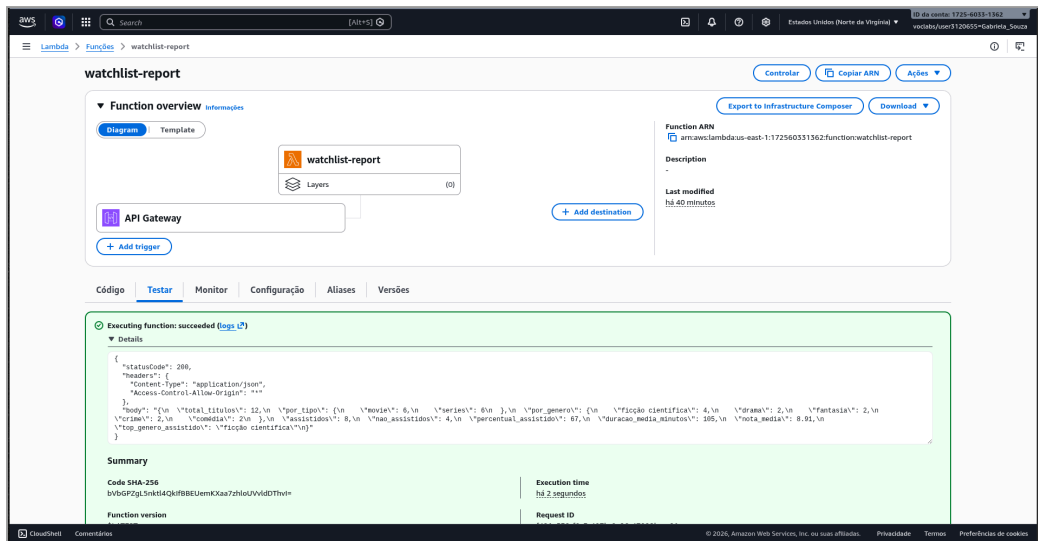


Figura 8 – Função de relatório e o resultado de um teste.

4.6 API Gateway

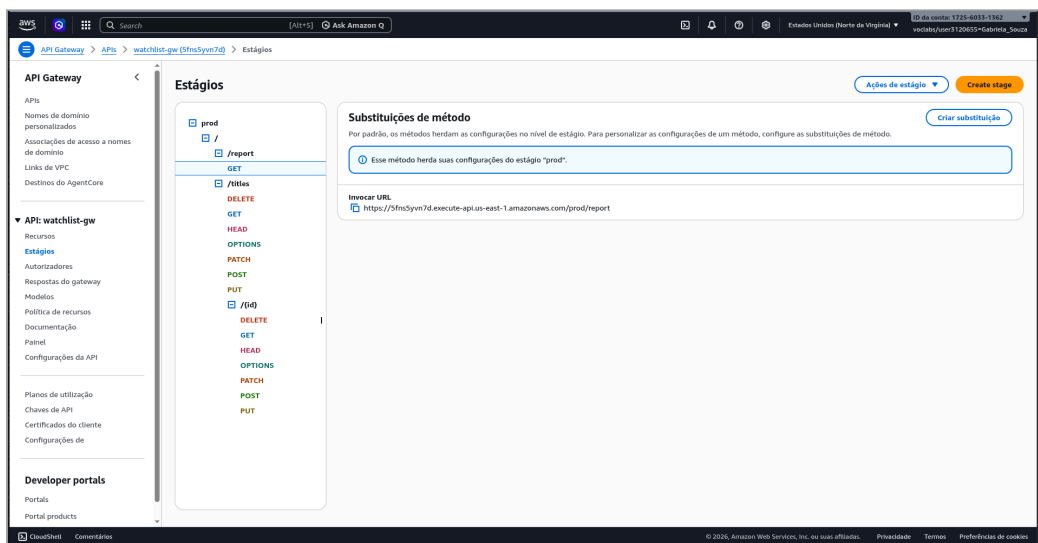


Figura 9 – Rotas /titles, /titles/{id} e /report publicadas.

5. A aplicação funcionando

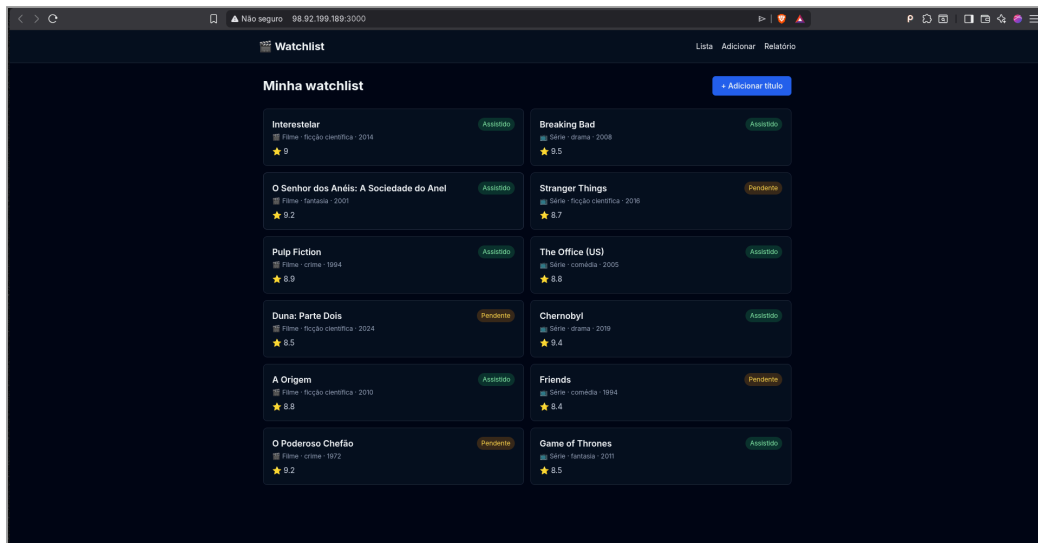


Figura 10 – Tela inicial com a lista de títulos.

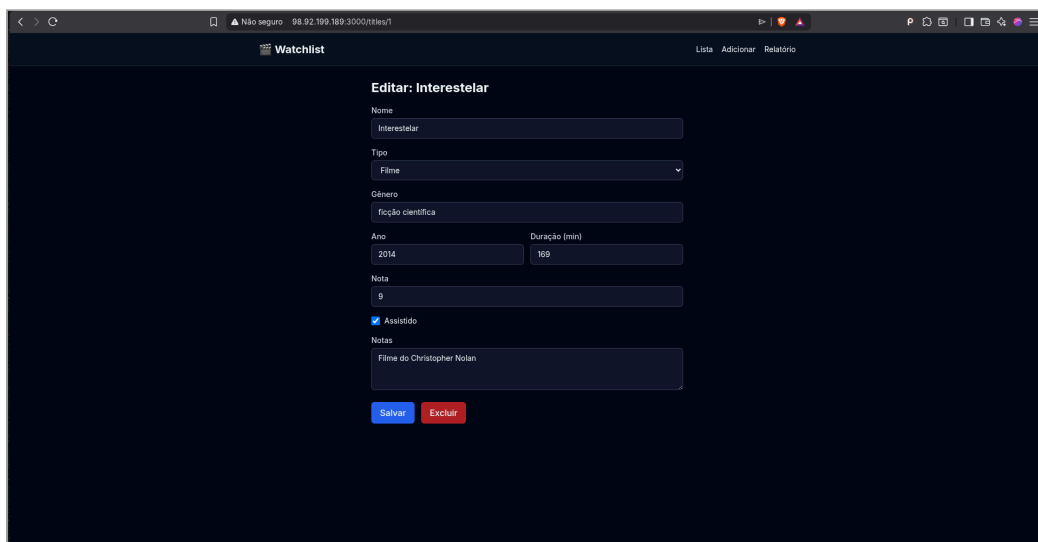


Figura 11 – Cadastro/edição de um título.

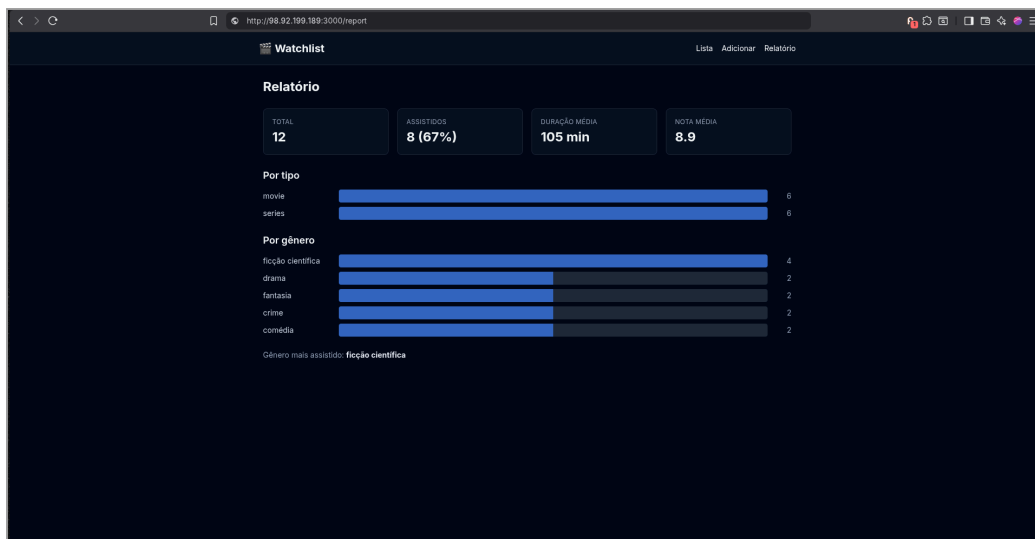


Figura 12 – Tela de relatório, com os dados vindos da Lambda.

6. Participação

O trabalho foi feito individualmente. A divisão das etapas ficou assim:

Etapa	Responsável
Back-end e modelo de dados	Gabriela Nellessen de Sousa
Front-end	Gabriela Nellessen de Sousa
Função Lambda	Gabriela Nellessen de Sousa
Configuração da AWS (VPC, RDS, EC2, Lambda, API Gateway)	Gabriela Nellessen de Sousa
Documentação e vídeo	Gabriela Nellessen de Sousa

7. Conclusão

Ver o app funcionando de ponta a ponta foi a parte mais satisfatória: abrir a tela, criar um título e acompanhar ele saindo do navegador, passando pelo API Gateway, chegando no back-end e sendo salvo no banco privado.

O que mais aprendi foi a parte de rede, que era justamente a que eu menos dominava — VPC, sub-redes e principalmente os Security Groups, que são o que mantém o banco protegido. Também entendi na prática a diferença entre rodar algo em contêiner e usar uma função serverless, já que deu pra ver as duas no mesmo projeto. Se eu fosse continuar, a primeira coisa que mudaria seria usar um domínio com HTTPS no lugar de acessar pelo IP, que hoje fica pouco amigável.